



The ReAct Loop

The core of every AI agent

SJSU · Edge AI

Reason + Act: an LLM in a loop with tools — run it as `sjsujetsontool chat --agent`.

1 Chatbot vs. agent

Chatbot — one prompt → one answer.
Knows only its weights + your prompt.

Agent — a chatbot put in a **loop** with **tools**.
Given a goal, it repeatedly decides *what to do next*, acts, observes, and continues.

Two ingredients turn a model into an agent:

1. **Tools** — functions it may call (read / grep / search / write / edit).
2. **A control loop** — code that runs the tool and feeds the result back.

That loop is the whole idea. RAG, coding assistants, multi-agent systems — all variations on it.

2 ReAct = Reason + Act

The model interleaves reasoning and actions in a strict text protocol — **one step per turn**:

```
Thought: I should look at app.py first  
Action: read_file  
Action Input: {"path": "app.py", "end": 40}
```

Our code runs the tool and appends the result; the model reads it and continues:

```
Observation: 1 import requests ...  
Thought: I now know the answer  
Final Answer: app.py calls requests.get(url) with a caller-supplied URL.
```

Plain text → works on **any** chat model (even local base models) and the reasoning is visible to debug.

3 The tools – `edge_agent/tools.py`

Five verbs a human coder uses — every path **confined to a project root**:

Tool	Purpose
<code>read_file(path, start, end)</code>	read a slice, with line numbers
<code>grep(pattern, path, is_regex)</code>	search contents → <code>file:line: text</code>
<code>search_files(glob, dir)</code>	find files by name
<code>write_file(path, content)</code>	create / overwrite
<code>edit_file(path, old, new)</code>	replace one exact, unique snippet

`edit_file` refuses unless `old` matches **exactly once** → forces a `read_file` first.
`dispatch()` always returns a string, so a bad call becomes an `Observation`, not a crash.

4 The loop – `react_loop.py`

Decoupled from HTTP: you pass a `complete(messages) -> str` (llama.cpp / NVIDIA / OpenAI / Anthropic).

```
def run(self, task):
    messages = [{"role": "system", "content": REACT_SYSTEM},
                {"role": "user", "content": task}]
    for step in range(self.max_steps):
        text = self.complete(messages)           # 1) reason
        messages.append({"role": "assistant", "content": text})
        if (m := _FINAL.search(text)):           # 2) done?
            return m.group(1).strip()
        act = _ACTION.search(text)                # 3) parse Action + JSON
        obs = self.tools.dispatch(act.group(1).strip(), json.loads(act.group(2))) # 4) act
        messages.append({"role": "user", "content": "Observation: " + obs})      # 5) observe
```

reason → done? → parse → act → observe, with a `max_steps` cap so it can't loop forever.

5 Run it

```
sjsujetsontool chat --agent --agent-dir ./sample_project
# ...or inside a normal chat session:
/agent on
/agent dir ./sample_project
What does app.py do, and is the requests CVE reachable?
```

Watch the trace stream by:

```
[step 1] Thought: read the app  Action: read_file  {"path":"app.py","end":40}
  Observation: 1  import requests ...
[step 2] Thought: confirm URL  Action: grep  {"pattern":"requests.get"}
  Observation: app.py:37: response = requests.get(url, timeout=5)
🤖 requests.get(url) takes a caller-supplied URL → the HTTP CVE path is reachable.
```

6 Two ways to call tools

	ReAct text loop	Native tool-calling
File	<code>react_loop.py</code>	<code>tool_calling.py</code>
Works on any chat model	✓ even base/local	✗ needs a tool model
Reasoning visible	✓	partly
Parsing	regex	provider-enforced JSON
Used by	<code>chat --agent</code> , CVE lab 12c	CVE lab 12b , Next.js API

Same `tools.py` — only the transport differs. This loop is the connective tissue across the labs.

7 Extend it — a one-function change

1. **Add a tool** → a method on `Tools` + its name in `TOOL_NAMES` (+ one `OPENAI_SCHEMAS` entry). Usable instantly.
2. **Swap the brain** → pass any `complete()` ; switch models with `/server` .
3. **Change the policy** → edit `REACT_SYSTEM` : require a plan first, or forbid `write_file` (a read-only auditor).
Full lesson → lkk688.github.io/edgeAI/curriculum/13_react_agent